

Chomsky (1956) §3.5

句構造文法の表現力

荒木理求

May 29, 2026

§3.5 の目標

§3.5 では、ここまでに定式化した 3 種類の言語クラスの包含関係を整理する:

- ▶ finite-state language
- ▶ terminal language
- ▶ derivable language

結論は、finite-state grammar よりも phrase-structure grammar のほうが本質的に強い、ということである。現代的な解釈では、「正規言語より文脈自由/依存言語のほうが表現力が高い」ことを意味する。ここで phrase-structure grammar (句構造文法) の用法が、計算機科学のものと異なることに注意されたい。

§3.4 の問題意識

§3.2 の句構造文法では, 規則は

$$X \rightarrow Y$$

の形であり, 左辺 X は単一記号でなければならない.

- ▶ これは句構造図が必ず構成できるようにするための制限である. (文脈自由言語に対応)
- ▶ しかし実際の文法記述では, 文脈ありの規則

$$Z \frown X \frown W \rightarrow Z \frown Y \frown W$$

を許すと簡単になる. (文脈依存言語に対応)

定義^{ix} : proper grammar

文法 $[\Sigma, Q]$ が proper grammar であるとは, 各初期列 X_i について, 少なくとも 1 つの規則

$$X_i \rightarrow Y_i$$

が存在し, その規則が obligatory であることをいう.

- ▶ つまり, 終端しない導出では, 毎回少なくとも 1 ステップ進まなければならない.
- ▶ proper grammar と一般の $[\Sigma, F]$ grammar の記述力は包含関係について比較不能である:

$$D_p \not\subseteq D_Q, \quad D_Q \not\subseteq D_p.$$

Chomsky(1956) は次の 3 種類の言語を導入した:

1. finite-state language: ある有限状態文法で生成される言語.
2. terminal language: ある句構造文法の終端列全体として得られる言語. すなわちそれ以上変形規則を適用できない (i.e. 導出が停止する) 語の集まりのこと.
3. derivable language: ある句構造文法から導出可能な列全体として得られる言語. 変形規則を適用する余地があっても, その段階を語として含めてよいということ.

定理 1x: 言語クラスの関係 (27)

次が成り立つ.

1. every finite-state language is a terminal language, but not conversely.
2. every derivable language is a terminal language, but not conversely.
3. there are derivable, nonfinite-state languages and finite-state, nonderivable languages.

証明 (1) (finite-state $\Rightarrow$ terminal) 有限状態文法 G が与えられたとする.

アイデア

有限状態の遷移を, 句構造文法の変形規則に変換する.

G で

$$(S_i, S_j) \in C$$

という遷移があれば, 句構造文法に

$$S_i \rightarrow a_{ijk} S_j$$

を入れる. また, 終了状態に対応して

$$S_i \rightarrow a_{i0k}$$

を入れる.

terminal $\Rightarrow$ finite-state の反例は簡単に見つかる:

$$L_1 = \{a^n b^n \mid n \geq 1\}$$

は finite-state language ではないが, terminal language である.
実際, 次の句構造文法で生成できる:

$$\Sigma : Z, \quad F : \begin{cases} Z \rightarrow a \frown b, \\ Z \rightarrow a \frown Z \frown b. \end{cases}$$

- ▶ $Z \rightarrow a \frown b$ で $n = 1$.
- ▶ $Z \rightarrow a \frown Z \frown b$ で外側に a, b を1つずつ接続させる.

finite-state grammar の特性

有限状態文法は、すでに読んだ a の個数を無制限には記憶できない。

- ▶ $a^n b^n$ では、前半の a の個数と後半の b の個数を一致させる必要がある。
- ▶ n は任意に大きい。
- ▶ したがって、有限個の状態だけでは必要な情報を保持できない。



ここでは mirror-image 的な依存関係が本質的である。

(2) (derivable $\Rightarrow$ terminal)

アイデア

derivable language を, 記号の読み替えを追加して terminal language として表す.

語彙を

$$V_p = \{a_1, \dots, a_n\}$$

とし, 新しい記号 b_i を用意する. さらに

$$a_i \rightarrow b_i \quad (1 \leq i \leq n)$$

を文法に追加する.

- ▶ b_i は元の語彙 V_p には含まれないとする.
- ▶ 導出可能な列を, 終端列として符号化する.

terminal $\Rightarrow$ derivable の反例: L_5 は次の列のみを含む言語であるとする:

$$a \wedge b, \quad c \wedge a \wedge b \wedge d, \quad c \wedge c \wedge a \wedge b \wedge d \wedge d, \quad \dots$$

つまり

$$L_5 = \{c^n a b d^n \mid n \geq 0\}.$$

これは terminal language である.

$$\Sigma : Z, \quad F : \begin{cases} Z \rightarrow a \wedge b, \\ Z \rightarrow c \wedge Z \wedge d. \end{cases}$$

アイデア

derivable language が無限集合なら, その中の無限個の列は, ある 1 つの規則の反復によって得られるはずである.

しかし, §3.2 の規則では, 書き換えは

$$X \rightarrow Y$$

であり, X は単一記号でなければならない.

- ▶ 1 回の規則適用で置き換えられるのは, 1 つの記号だけである.
- ▶ $c^n abd^n$ のように, 左端側と右端側を同時に伸ばすには, 1 つの記号を書き換えるだけでは足りない.



L_5 は terminal にはできるが, derivable そのものにはできない.

結論

1. finite-state language は terminal language に含まれる.
2. derivable language も terminal language に含まれる.
3. しかし finite-state と derivable は比較不能である.
4. phrase-structure grammar は, finite-state grammar より抽象的で強力である.

!

Chomsky (1956) の大きな流れでは, finite-state model の限界を示したあと, より強い phrase-structure model へ進むための基礎づけになっている.